SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

그래프의 기본과 탐색

Python

학습시작

한 도시에서 전염병이 시작됐습니다.

도시마다 사람들의 왕래가 많아 전염병이 퍼질 위험이 있습니다.
아래 질문에 답해 봅시다.

1. 모든 도시가 전염될 수 있는지 확인하려면?
2. 가장 빨리 전염되는 도시 순서를 알고 싶다면?

- 도시, 네트워크, 친구 관계처럼 서로 연결된 대상의 관계를 표현하고 분석할 수 있습니다.
- 연결 정보를 표나 목록으로 저장하고 관리하는 방법을 익힐 수 있습니다.
- 한 경로를 끝까지 탐험하거나, 가까운 곳부터 차례대로 탐색하는 과정을 직접 구현할 수 있습니다.
- 서로 연결된 그룹을 찾아내고, 왕래가 전혀 없는 영역을 구분할 수 있는 방법을 적용할 수 있습니다.

학습목표

- 현실 세계 문제를 그래프로 추상화해서 해결하는 방법을 학습한다.
- 그래프 탐색 기법인 BFS와 DFS에 대해 학습한다.
- 그래프 알고리즘에 활용되는 상호 배타 집합(Disjoint Set)의 자료구조에 대해 학습한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

그래프의 기본과 탐색

- 그래프 기본
- DFS
- BFS
- Union-Find(Disjoint Set)

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

그래프의 기본과 탐색

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

그래프 기본

문제 제시 : 친구 관계

- A의 친구는 B : (A - B)
- C의 친구는 E, F : (C - E, F)
- (D - E), (F - G), (N - B, I, L), (G - A, C, D, H), (I - J, H), (B - D, K, L), (M - I, J), (E - A, H), (C - B, I, L), (B - I), (J - A, G) 처럼 친구 관계가 주어짐

- A의 친구 중에 친구가 가장 많은 친구는 누구인가?

그래프

그래프

아이템(사물 또는 추상적 개념)들과 이들 사이의 연결 관계를 표현

선형 자료구조나 트리 자료구조로 표현하기 어려운 N : N 관계를 가지는 원소들을 표현하기에 용이합니다.

그래프

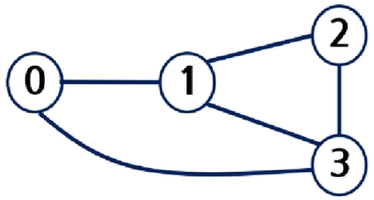
- 정점(Vertex)들의 집합과 이들을 연결하는 간선(Edge)들의 집합으로 구성된 자료 구조

- |V| : 정점의 개수, |E| : 그래프에 포함된 간선의 개수라 할 때,
  |V| 개의 정점을 가지는 그래프는 최대 |E| = |V|*(|V| - 1)/ 2개의 간선을 가질 수 있습니다.

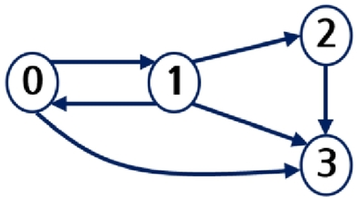
- 예) 5개 정점이 있는 그래프의 최대 간선 수는 10(= 5*4/2)개 입니다.

그래프 유형

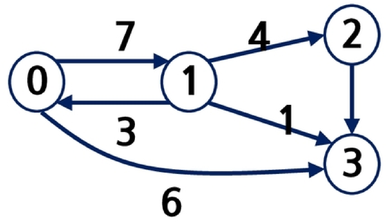
- 무향(무방향) 그래프(Undirected Graph)
- 유향(방향) 그래프(Directed Graph)
- 가중치 그래프(Weighted Graph)
- 사이클 없는 방향 그래프(DAG, Directed Acyclic Graph, 방향성 비순환 그래프)



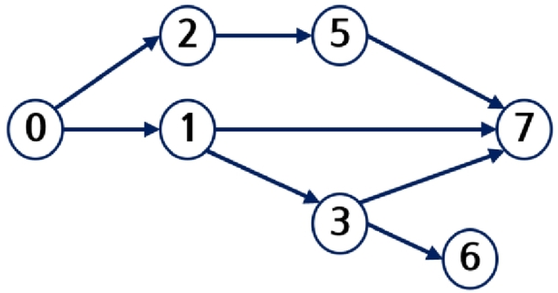
<그림1_알고리즘 응용_그래프 기본과 탐색_무향 그래프>



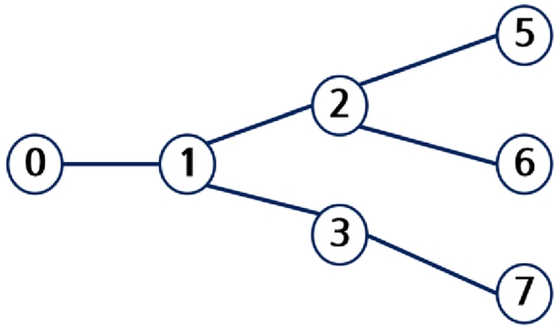
<그림2_알고리즘 응용_그래프 기본과 탐색_유향 그래프>



<그림3_알고리즘 응용_그래프 기본과 탐색_가중치 그래프>



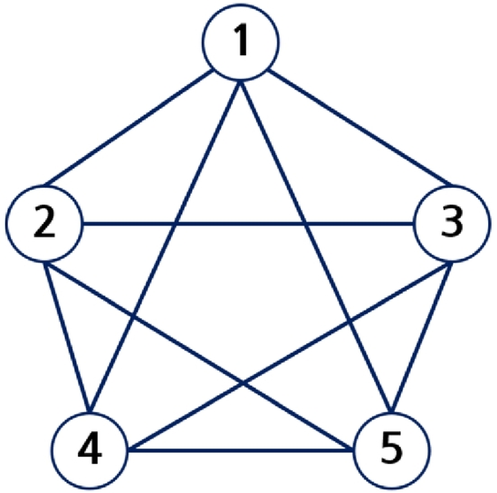
<그림4_알고리즘 응용_그래프 기본과 탐색_사이클 없는 방향 그래프>



<그림5_알고리즘 응용_그래프 기본과 탐색_사이클이 없는 무향 그래프>

그래프 유형

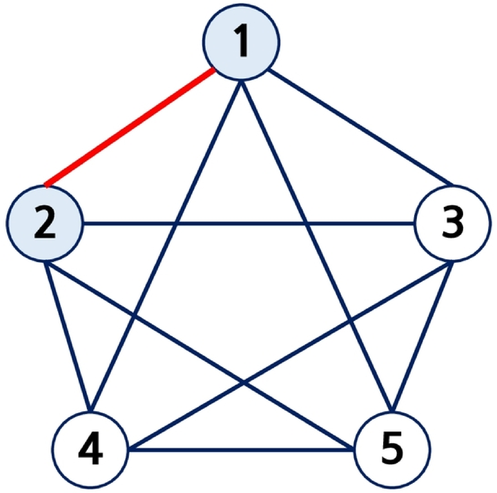
- 완전 그래프: 정점들에 대해 가능한 모든 간선들을 가진 그래프
  - 그림6의 경우 5개 정점이 있으므로 최대 간선 수인 10개의 간선을 갖습니다.
- 부분 그래프: 원래 그래프에서 일부의 정점이나 간선을 제외한 그래프



<그림6_알고리즘 응용_그래프 기본과 탐색_완전 그래프>

인접 정점

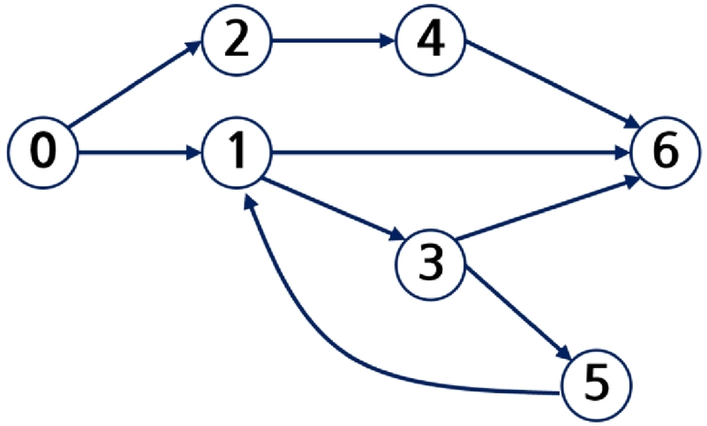
- 인접(Adjacency)
  - 두 개의 정점에 간선이 존재(연결됨)하면 서로 인접 관계라고 합니다.
  - 완전 그래프에 속한 임의의 두 정점들은 모두 인접해 있습니다.



<그림7_알고리즘 응용_그래프 기본과 탐색_인접>

그래프 경로

- 경로란 간선들을 순서대로 나열한 것
  - 예) 정점 0에서 정점 6으로 가는 경로
    간선들로 표현 : (0, 2), (2, 4), (4, 6)
    정점들로 표현 : 0 - 2 - 4 - 6
- 경로 중 한 정점을 최대한 한 번만 지나는 경로를 단순경로라고 함
  0 - 2 - 4 - 6, 0 - 1 - 6
- 시작한 정점에서 끝나는 경로를 사이클(Cycle)이라고 함
  1 - 3 - 5 - 1



<그림8_알고리즘 응용_그래프 기본과 탐색_그래프 경로>

그래프 표현

- 간선의 정보를 저장하는 방식, 메모리나 성능을 고려해서 결정
- 인접 행렬 (Adjacency Matrix)
  - |V| x |V| 크기의 2차원 배열을 이용해서 간선 정보를 저장합니다.

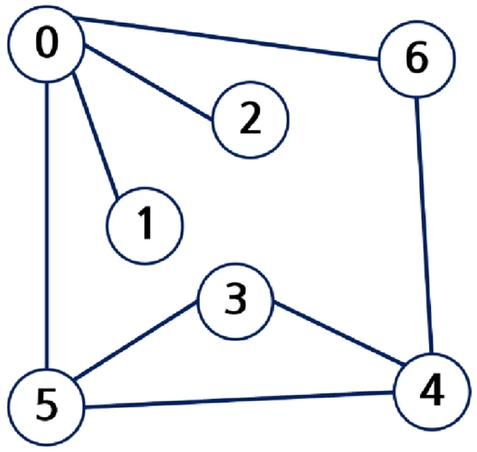
- 인접 리스트 (Adjacency List]
  - 각 정점마다 해당 정점과 인접한 정점 정보를 저장

- 간선의 배열
  - 간선(시작 정점, 끝 정점)을 배열에 연속적으로 저장

인접 행렬:두 정점을 연결하는 간선의 유무를 행렬로 표현

- |V| x |V| 정방 행렬
- 행 번호와 열 번호는 그래프의 정점에 대응
- 두 정점이 인접되어 있으면 1, 그렇지 않으면 0으로 표현
- 무향 그래프

 - i 번째 행의 합 = i 번째 열의 합 = Vi 의 차수

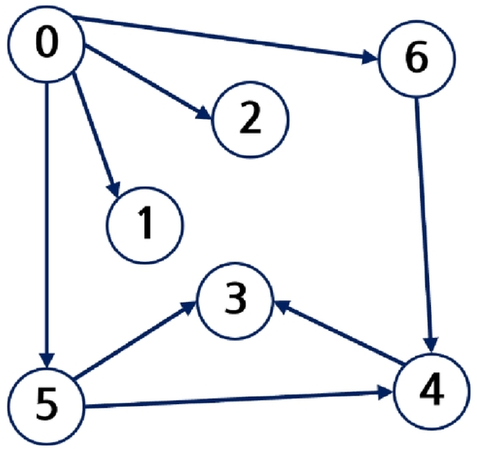


	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1	1	0	0	0	0	0	0
2	1	0	0	0	0	0	0
3	0	0	0	0	1	1	0
4	0	0	0	1	0	1	1
5	1	0	0	1	1	0	0
6	1	0	0	0	1	0	0

<그림9_알고리즘 응용_그래프 기본과 탐색_인접 행렬>

인접 행렬:두 정점을 연결하는 간선의 유무를 행렬로 표현

- 유향 그래프
  - 행 i 의 합 = Vi 의 진출 차수
  - 열 i 의 합 = Vi 의 진입 차수



	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	1	0	0	0
5	0	0	0	1	1	0	0
6	0	0	0	0	1	0	0

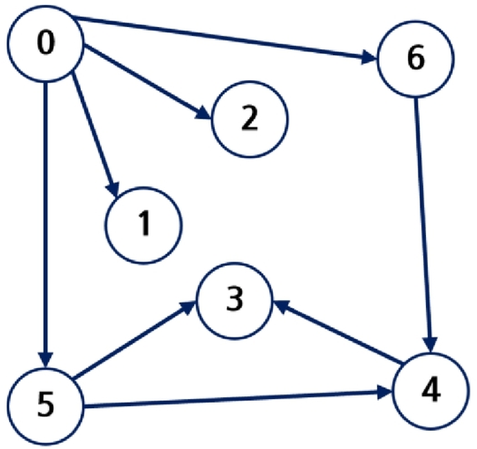
→ 진출차수

↓ 진입차수

<그림10_알고리즘 응용_그래프 기본과 탐색_진입차수와 진출차수>

인접 행렬의 단점은?

- 정점 수에 비해 간선 수가 적은 경우 공간 낭비가 심함



	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	1	0	0	0
5	0	0	0	1	1	0	0
6	0	0	0	0	1	0	0

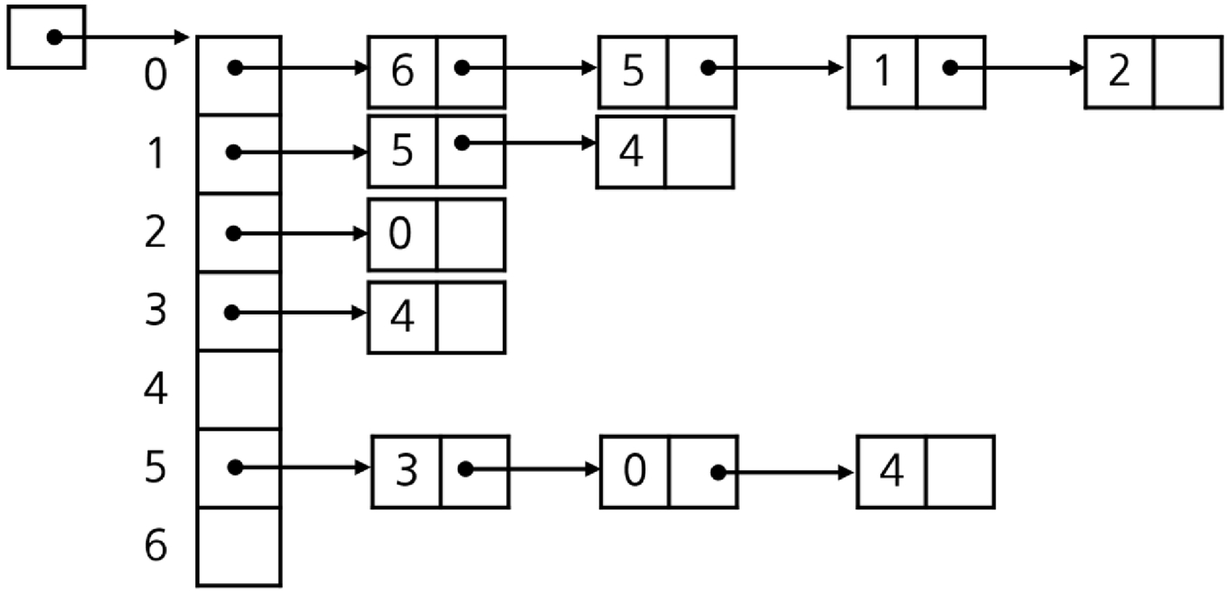
	시작 정점	끝 정점
0	0	1
1	0	2
2	0	5
3	0	6
4	4	3
5	5	3
6	5	4
7	6	4

<그림11_알고리즘 응용_그래프 기본과 탐색_인접 행렬의 단점>

인접 리스트

- 각 정점에 대한 인접 정점들을 순차적으로 표현

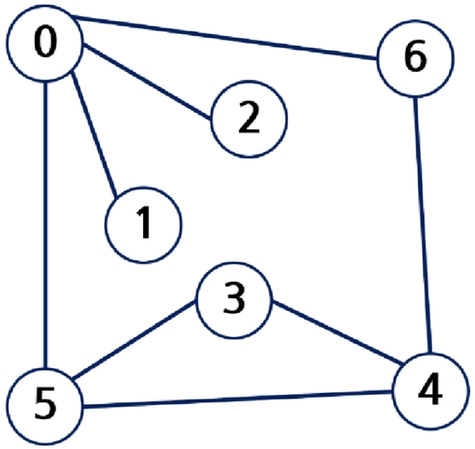
- 하나의 정점에 대한 인접 정점들을 각각 노드로 하는 연결 리스트로 저장

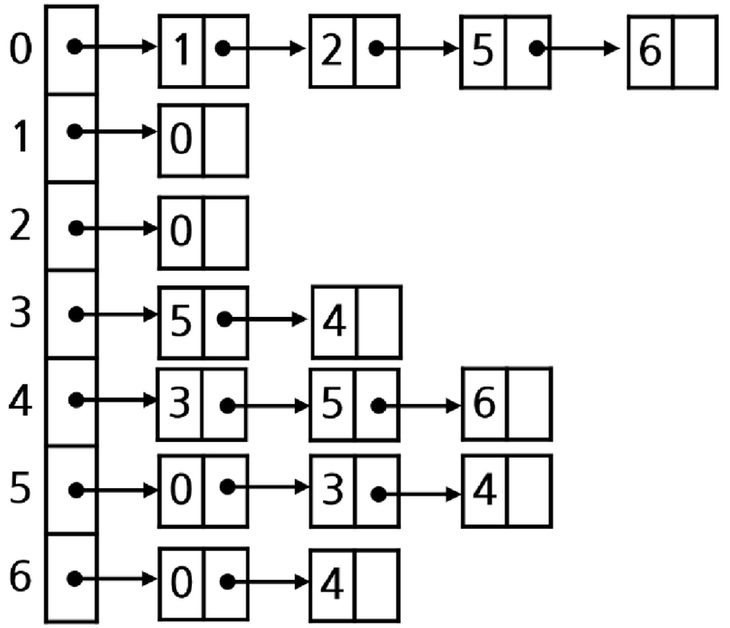


〈그림12_알고리즘 응용_그래프 기본과 탐색_인접 리스트〉

인접 리스트

- 무향 그래프의 인접 리스트





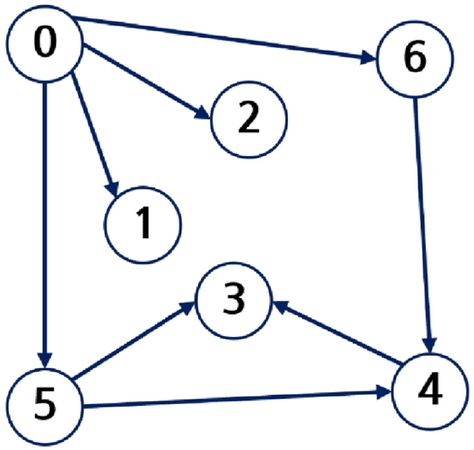
<그림13_알고리즘 응용_그래프 기본과 탐색_인접 리스트 - 무방향 그래프>

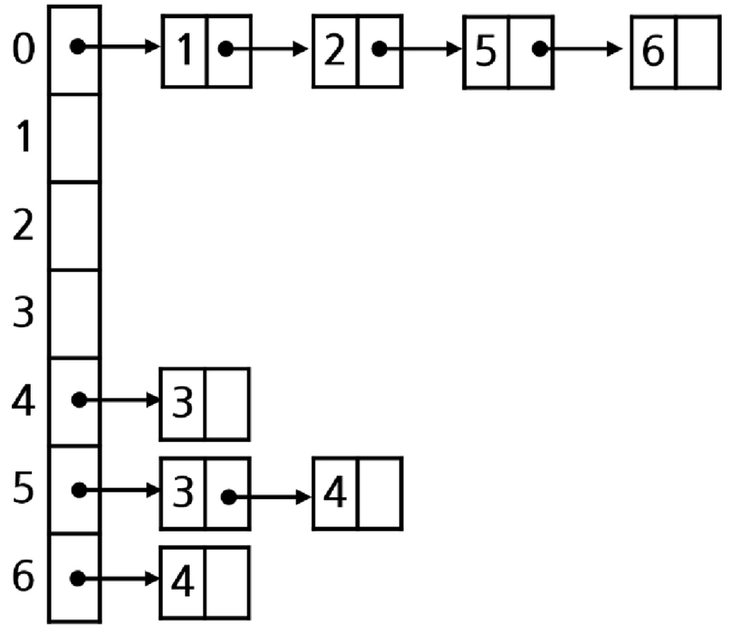
인접 리스트의 노드 수 = 간선의 수 * 2

각 정점의 노드 수 = 정점의 차수

인접 리스트

- 유향 그래프의 인접 리스트





<그림14_알고리즘 응용_그래프 기본과 탐색_인접 리스트 - 방향 그래프>

인접 리스트 노드 수 = 간선의 수

각 정점의 노드 수 = 정점의 진출 차수

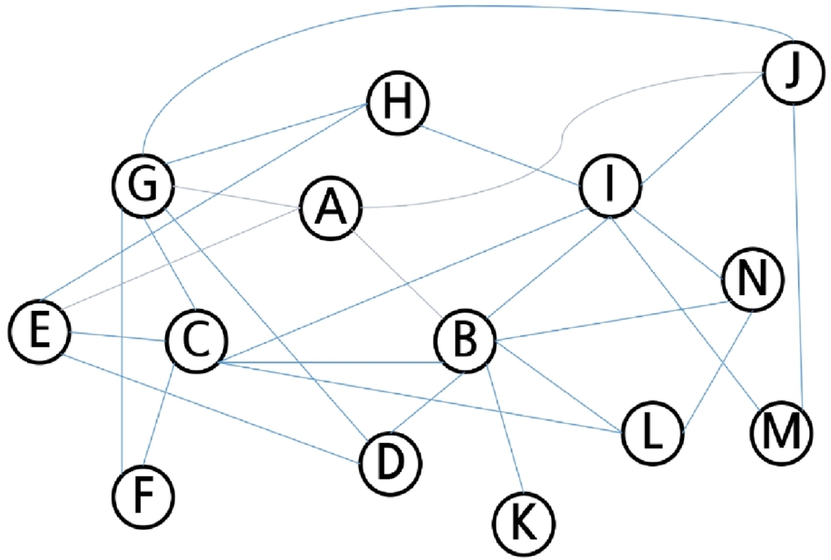
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DFS

문제 제시 : 친구 관계

- 다음과 같이 친구 관계를 그래프로 표현하였다.
- A로부터 시작해서 한 명의 친구에게만 소식을 이어서 전달할 수 있다면 최대 몇 명의 친구가 소식을 전달받을 수 있을까? (단, 소식을 전달받은 친구한테는 소식을 재전달할 수 없다.)
- A로부터 시작해서 친구들에게 동시에 소식을 전달할 수 있다고 할 때, 가장 늦게 전달 받는 사람은 누구일까? (단, 친구에게 소식을 전달하는 속도는 동일하다)



<그림15_알고리즘 응용_그래프 기본과 탐색_문제 - 친구 관계>

DFS (Depth First Search, 깊이 우선 탐색) 알고리즘 - 재귀

- 모든 정점을 중복 및 빠짐없이 방문하는 경우
  - G : 탐색할 그래프
  - v : 방문하는 정점

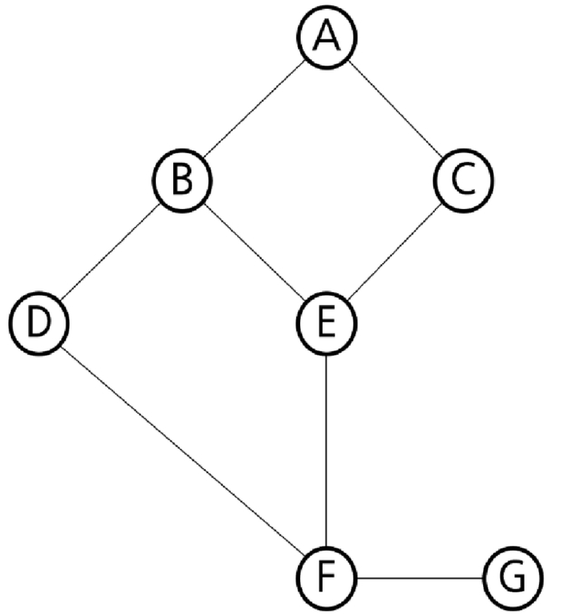
```
DFS_Recursive(G,  v)

    visited[ v ] ← TRUE                      # v 방문 설정

    FOR  each all w in adjacency( G, v )  # v에 인접한 정점 w에 대해
        IF  visited[w]  ≠  TRUE              # w에 방문한 적이 없으면
            DFS_Recursive(G, w)
```

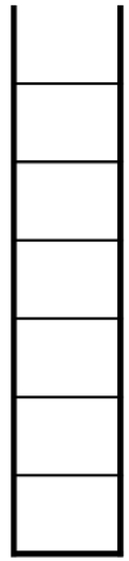
DFS 알고리즘 - 반복

```
STACK s                    # 스택 생성
visited[ ]                 # 방문한 정점 표시
DFS(v)
    push( s, v )                 # 시작 정점 push
    WHILE NOT isEmpty( s )
          v ← pop(s)
          IF NOT visited[v]
              visit( v )
              FOR each w in adjacency( v )
                    IF NOT visited[w]
            push(s, w)
```

DFS 예

```
STACK s
visited[ ]
DFS(v)
    push( s, v )
    WHILE NOT isEmpty( s )
        v ← pop(s)
        IF NOT visited[v]
            visit( v )
            FOR each w in adjacency( v )
                IF NOT visited[w]
                    push(s, w)
```





정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	F	F	F	F	F	F	F

<그림16_알고리즘 응용_그래프 기본과 탐색_DFS 예>

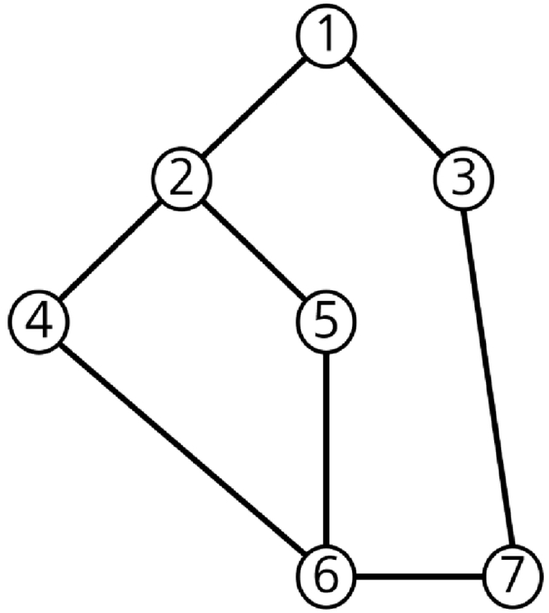
중복검사가 필요 없는 DFS

- 스택에 push된 정점을 visited에 표시함

```
STACK s
visited[ ]
DFS(v)
    push( s, v )
    visited[v] = True
    WHILE NOT isEmpty( s )
        v ← pop(s)
        visit( v )
        FOR each w in adjacency( v )
            IF NOT visited[w]
                push(s, w)
                visited[v] = True
```

연습

- 1번 정점부터 깊이 우선 탐색하는 경우 정점 방문 순서
  - 입력 : 정점 수, 간선 수와 각 간선의 끝 정점
    7 8
    1 2 1 3 2 4 2 5 4 6 5 6 6 7 3 7
  - 방문 순서의 예 (구현 방법에 따라 정점 순서는 달라질 수 있음)
    1 2 4 6 5 7 3



〈그림17_알고리즘 응용_그래프 기본과 탐색_탐색 연습〉

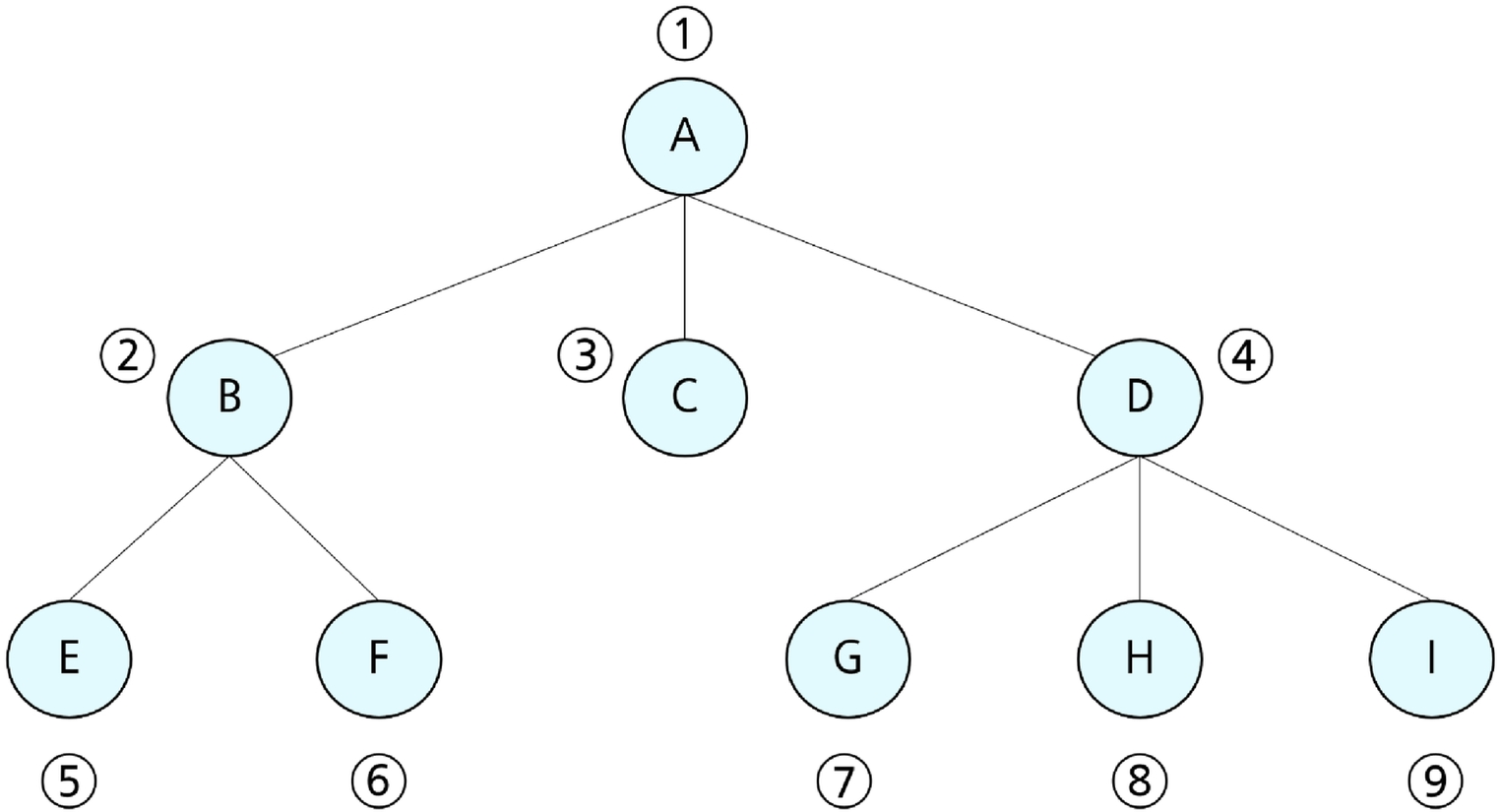
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

BFS

BFS (Breadth First Search, 너비 우선 탐색) 알고리즘

- BFS는 예제 그래프를 붙여진 번호 순서로 탐색함



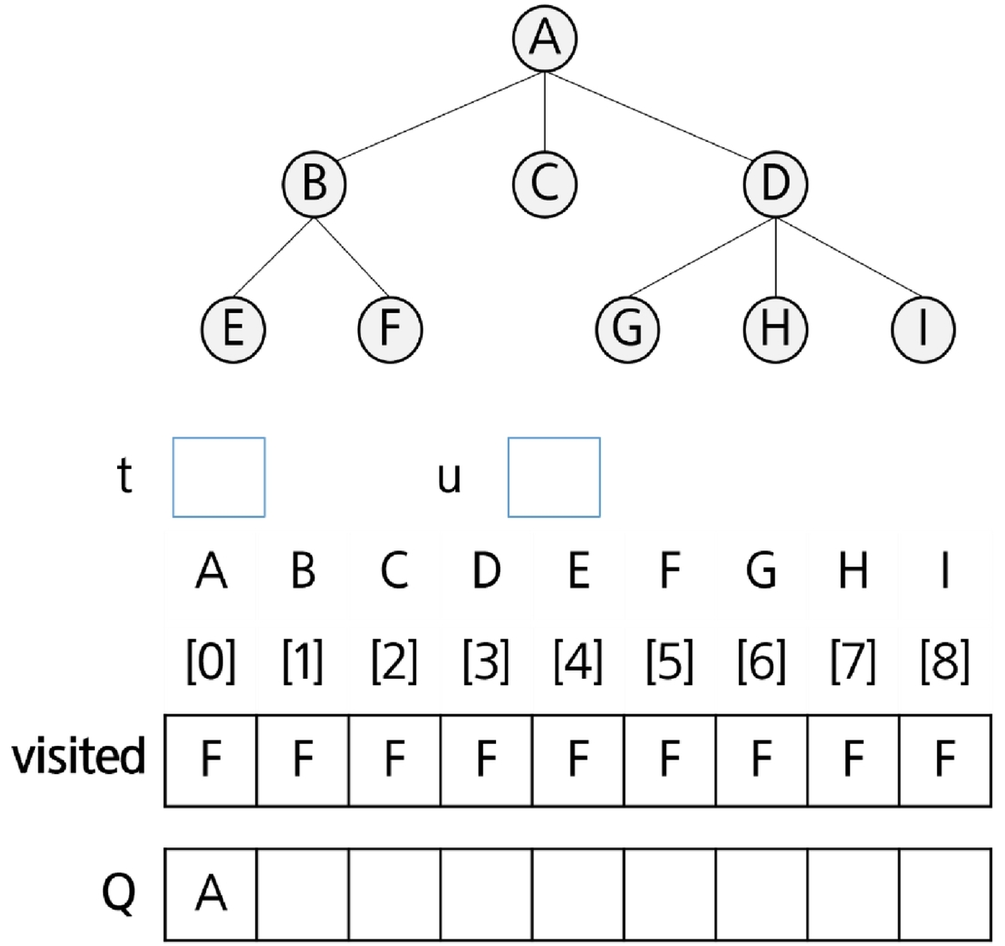
<그림18_알고리즘 응용_그래프 기본과 탐색_BFS 알고리즘>

입력 파라미터: 그래프 G와 탐색 시작점 v

```
BFS(G,v) // 그래프 G, 탐색 시작점 v
    큐 생성
    시작점 v를 큐에 삽입
    정점 v를 인큐된 것으로 표시
    WHILE 큐가 비어 있지 않은 경우
        t ← 큐의 첫 번째 원소 반환
        FOR t와 연결된 모든 선에 대해
            u ← t의 인접점
            u가 인큐되지 않은 곳이면,
            u를 큐에 넣고, 인큐된 것으로 표시
```

BFS 예

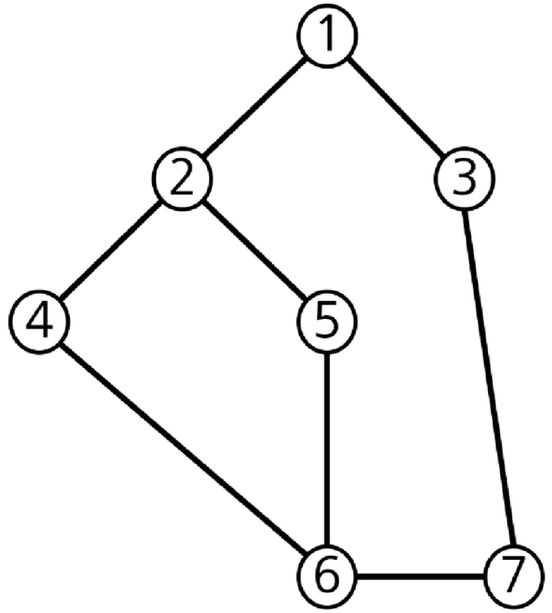
```
BFS(G,v) // 그래프 G, 탐색 시작점 v
    큐 생성
    시작점 v를 큐에 삽입
    정점 v를 인큐된 것으로 표시
    WHILE 큐가 비어 있지 않은 경우
        t ← 큐의 첫 번째 원소 반환
        FOR t와 연결된 모든 선에 대해
            u ← t의 인접점
            u가 인큐되지 않은 곳이면,
            u를 큐에 넣고, 인큐된 것으로 표시
```



<그림19_알고리즘 응용_그래프 기본과 탐색_BFS 예>

연습

- 1번 정점부터 너비 우선 탐색하는 경우 정점 방문 순서
  - 입력 : 정점 수, 간선 수, 각 간선의 끝 정점
    7 8
    1 2 1 3 2 4 2 5 4 6 5 6 6 7 3 7
  - 방문 순서의 예 (구현 방법에 따라 중간 정점 순서는 달라질 수 있음)
    1 2 3 4 5 7 6



<그림17_알고리즘 응용_그래프 기본과 탐색_탐색 연습>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Union-Find (Disjoint Set)

서로소 집합(Disjoint Set)

- 서로 공통 원소가 없는 집합
  - 교집합이 없는 집합들입니다.

- 대표자(representative)
  - 각 집합을 대표하는 하나의 원소를 말합니다.

- 상호 배타 집합
  - 확률, 논리, 집합론에서 동시에 일어날 수 없는 경우, 공통이 없는 경우를 말합니다.
  - 집합론에서는 서로소와 같은 뜻으로 사용됩니다.

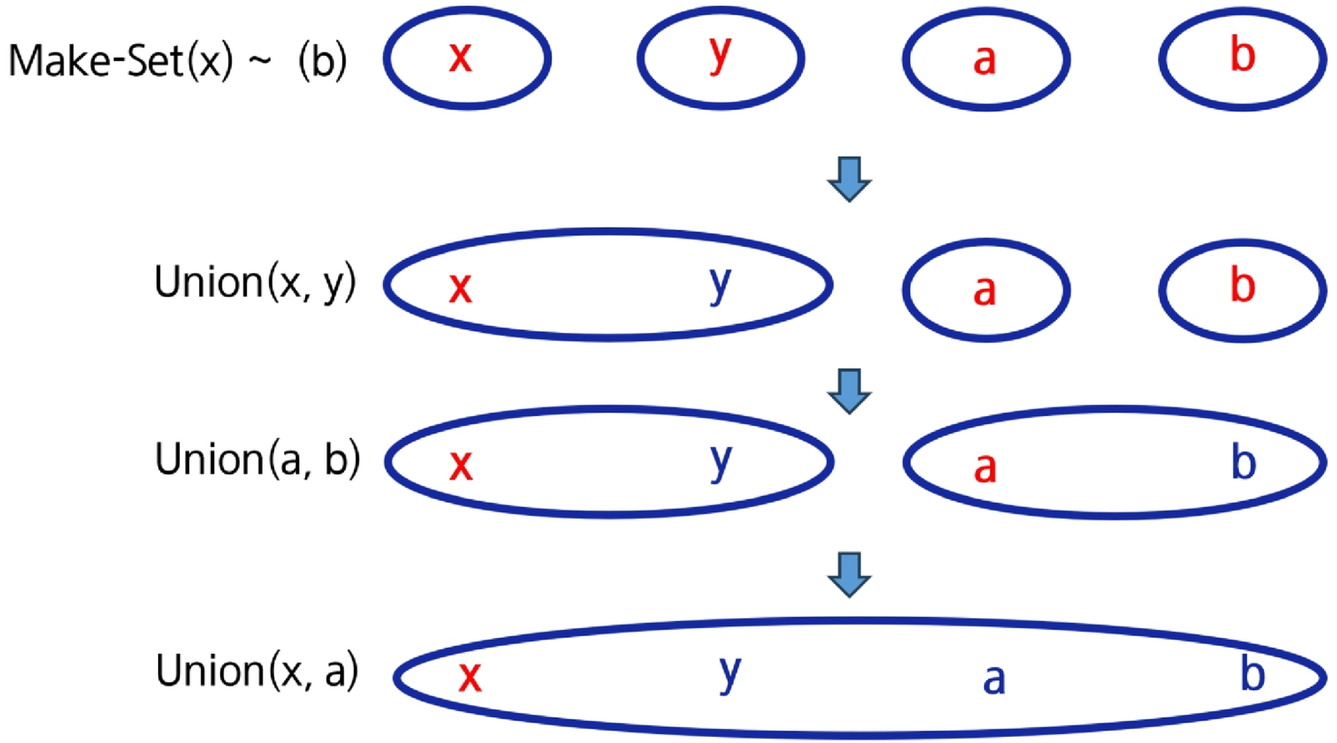
- 표현 방법
  - 연결리스트로 표현할 수 있습니다.
  - 트리를 이용해 표현할 수 있습니다.

서로소 집합 연산

- Make-Set(x)
  - x를 원소로 가진 집합을 만듭니다.
- Find-Set(x)
  - x가 속한 집합의 대표 원소를 반환합니다.
- Union(x , y)
  - y가 속한 집합과 x가 속한 집합의 합집합을 만듭니다.
  - x가 합집합의 대표 원소가 됩니다.

상호배타 집합 예

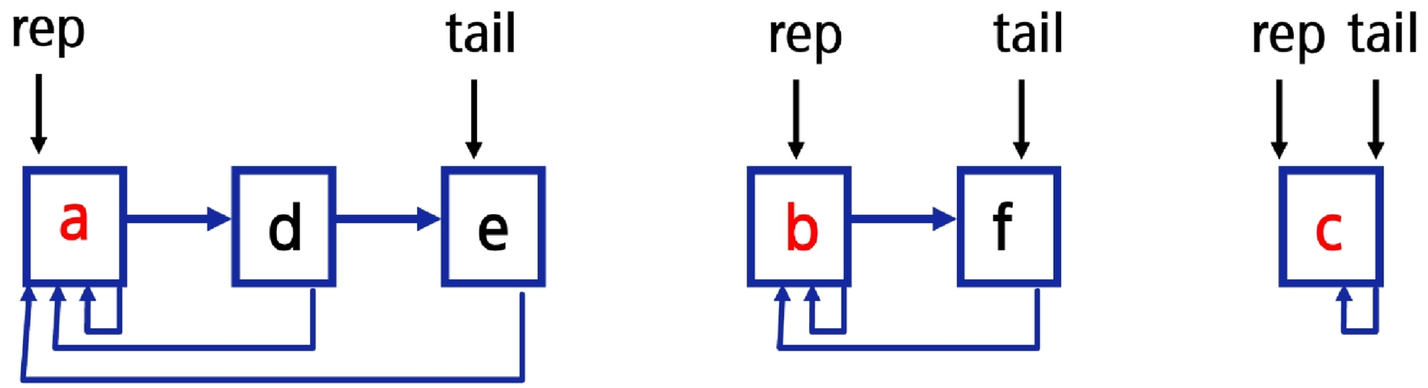
- Make-Set(x)
- Make-Set(y)
- Make-Set(a)
- Make-Set(b)
- Union(x, y)
- Union(a, b)
- Find-Set(y) -> x
- Find-Set(b) -> a
- Union(x, a)



<그림20_알고리즘 응용_그래프 기본과 탐색_상호 배타 집합 예>

상호 배타 집합 표현 - 연결리스트

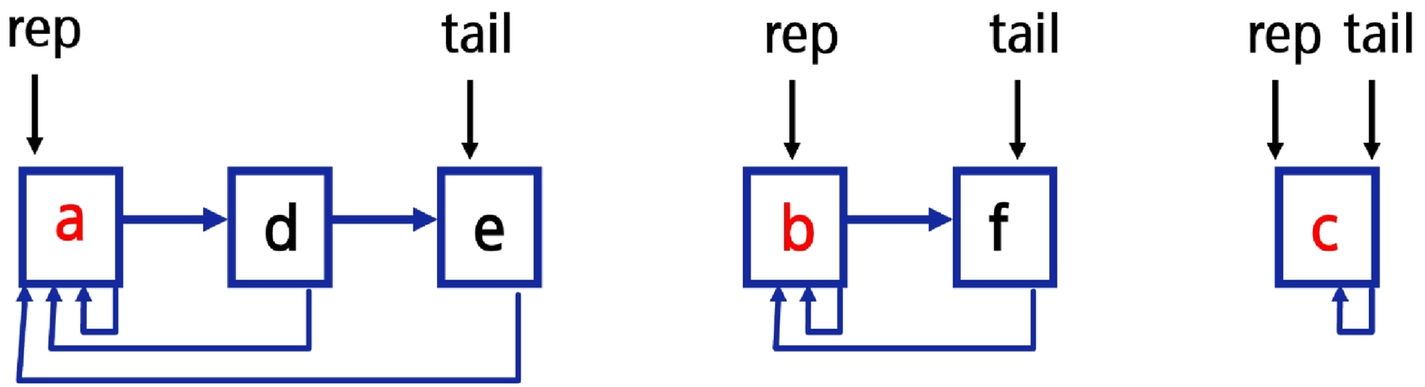
- 같은 집합의 원소들은 하나의 연결리스트로 관리
- 연결리스트의 맨 앞의 원소가 집합의 대표 원소
- 각 원소는 집합의 대표 원소를 가리키는 링크를 가짐



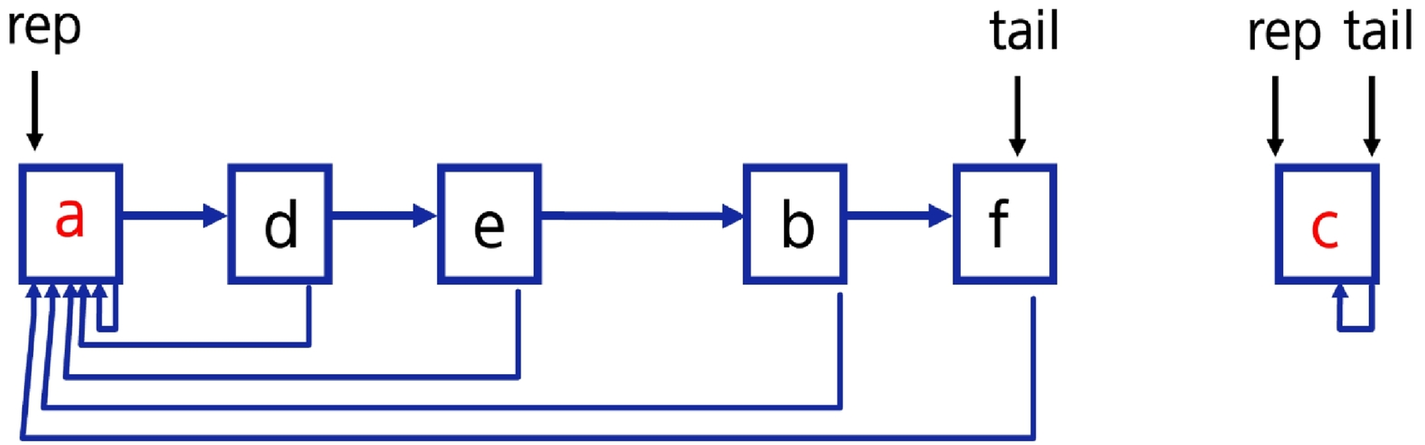
<그림21_알고리즘 응용_그래프 기본과 탐색_상호 배타 집합 표현 - 연결리스트>

연결리스트 연산 예

- Find-Set(e) return a
- Find-Set(f) return b
- Union(a, b)



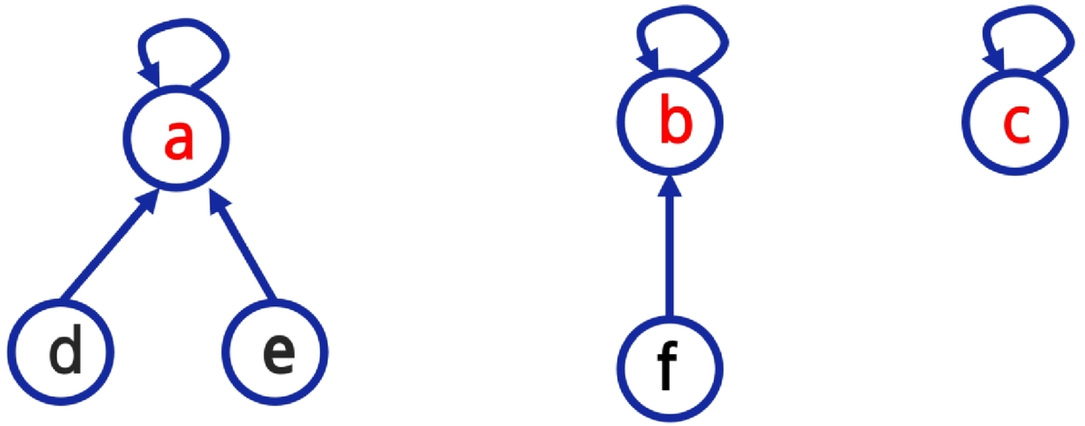
<그림21_알고리즘 응용_그래프 기본과 탐색_상호 배타 집합 표현 - 연결리스트>



<그림22_알고리즘 응용_그래프 기본과 탐색_연결리스트 연산 예>

상호 배타 집합 표현 - 트리

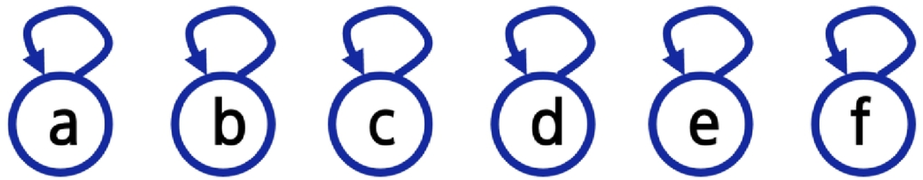
- 하나의 집합(a disjoint set)을 하나의 트리로 표현함
- 자식 노드가 부모 노드를 가리키며 루트 노드가 대표자가 됨



<그림23_알고리즘 응용_그래프 기본과 탐색_상호 배타 집합 표현 - 트리>

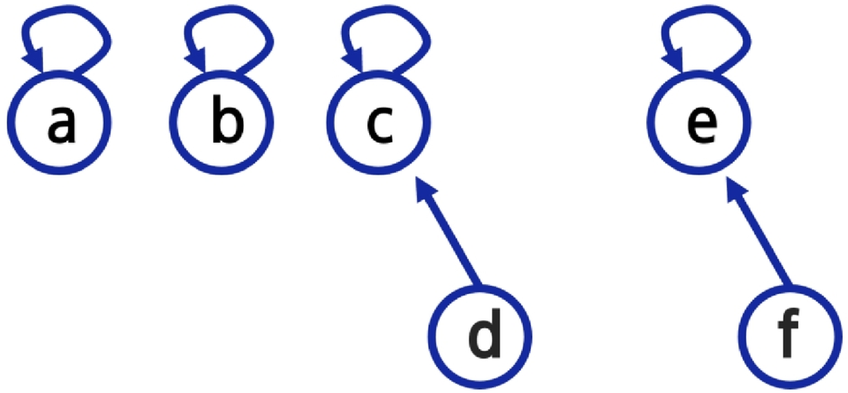
트리 연산 예

- Make-Set(a) ~ Make-Set(f)



<그림24_알고리즘 응용_그래프 기본과 탐색_트리 연산 예 - Make-Set>

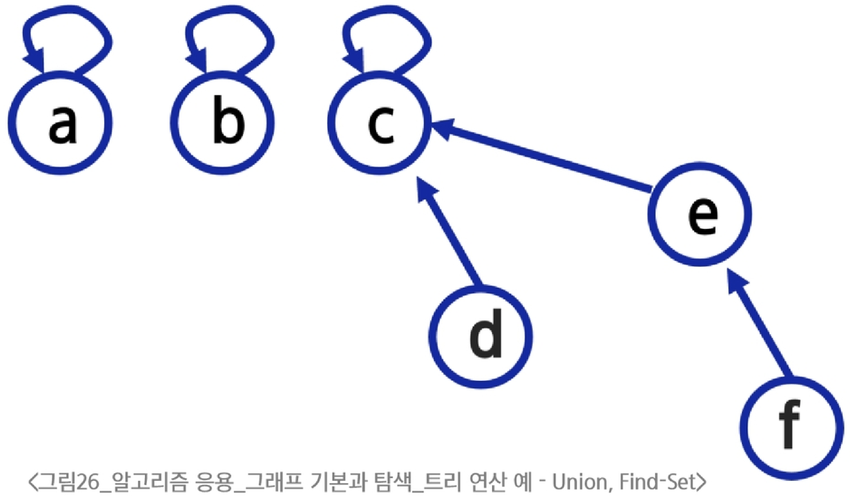
- Union(c, d), Union(e, f)



<그림25_알고리즘 응용_그래프 기본과 탐색_트리 연산 예 - Union>

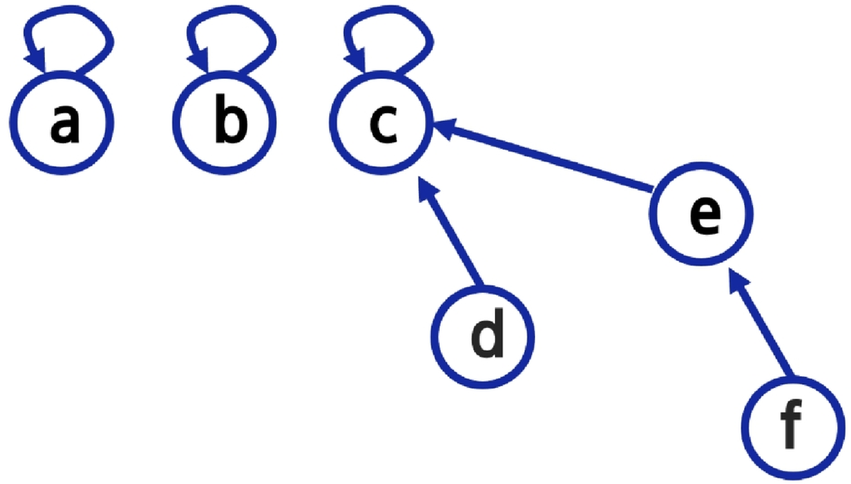
트리 연산 예

- Union(d, f)
- Find-Set(d) return c
- Find-Set(e) return c



<그림26_알고리즘 응용_그래프 기본과 탐색_트리 연산 예 - Union, Find-Set>

상호배타 집합을 표현한 트리의 배열을 이용한 저장



첨자	0	1	2	3	4	5
정점	a	b	c	d	e	f
부모	0	1	2	2	2	4

<그림27_알고리즘 응용_그래프 기본과 탐색_상호배타 집합을 표현한 트리의 배열>

상호 배타 집합에 대한 연산

- Make-Set(x): 유일한 멤버 x를 포함하는 새로운 집합을 생성하는 연산

```
Make-Set(x)
    p[x] ← x
```

- Find_Set(x): x를 포함하는 집합을 찾는 연산

```
Find-Set(x)
    IF x == p[x] : RETURN x
    ELSE         : RETURN Find_Set(p[x])
```

- Union(x , y): x와 y를 포함하는 두 집합을 통합하는 연산

```
Union(x, y)
    p[Find-Set(y)] ← Find-Set(x)
```

상호 배타 집합에 대한 연산

- Find_Set(x): x를 포함하는 집합을 찾는 연산 (반복)

```
Find-Set(x)
    while x != p[x]
        x = p[x]
    return x
```

상호 배타 집합에 대한 연산

- Make-Set(1)~(6)
- Union(1,3)
- Union(2,3)
- Union(5,6)
- 여기까지 연산 후 P 배열

Index	1	2	3	4	5	6
P	2	2	1	4	5	5

<표1_알고리즘 응용_그래프의 기본과 탐색_상호 배타 집합에 대한 연산>

- Find_Set(6)

 - 5가 반환됩니다.

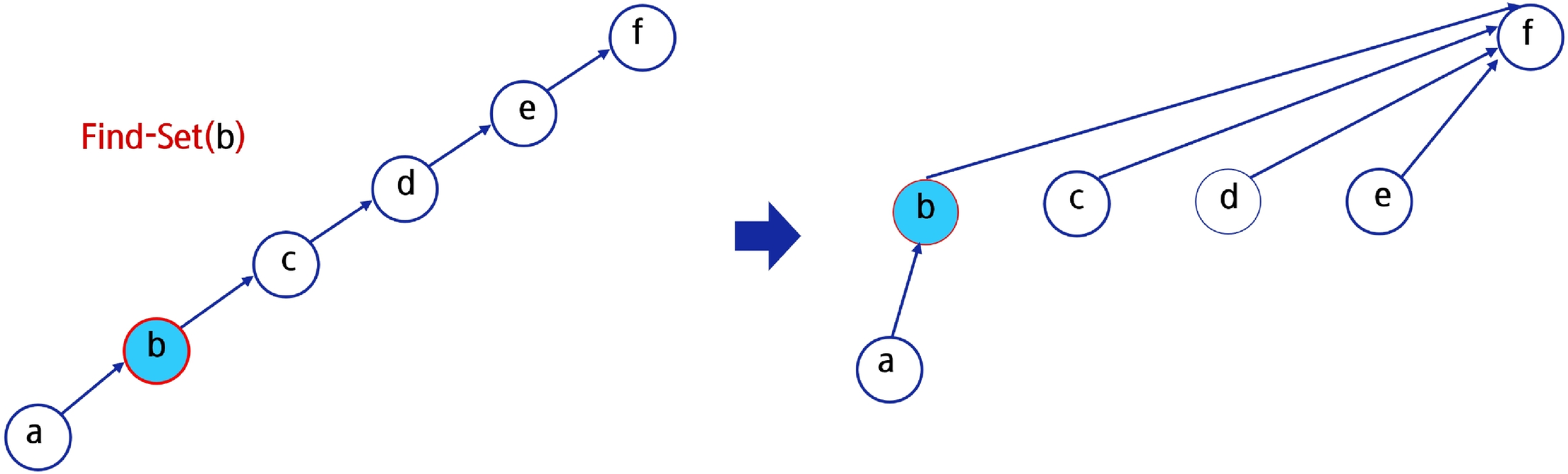
```
Make-Set(x)
    p[x] ← x
```

```
Find-Set(x)
    IF x == p[x] : RETURN x
    ELSE         : RETURN Find_Set(p[x])
```

```
Union(x, y)
    p[Find-Set(y)] ← Find-Set(x)
```

상호 배타 집합에 대한 연산 문제점

- 편향 트리가 되는 경우 Find_Set()에 시간이 오래 걸림
- Find_set()에서 찾아진 대표 원소로 각 원소의 대표 원소를 바꾸면 경로가 단축됨

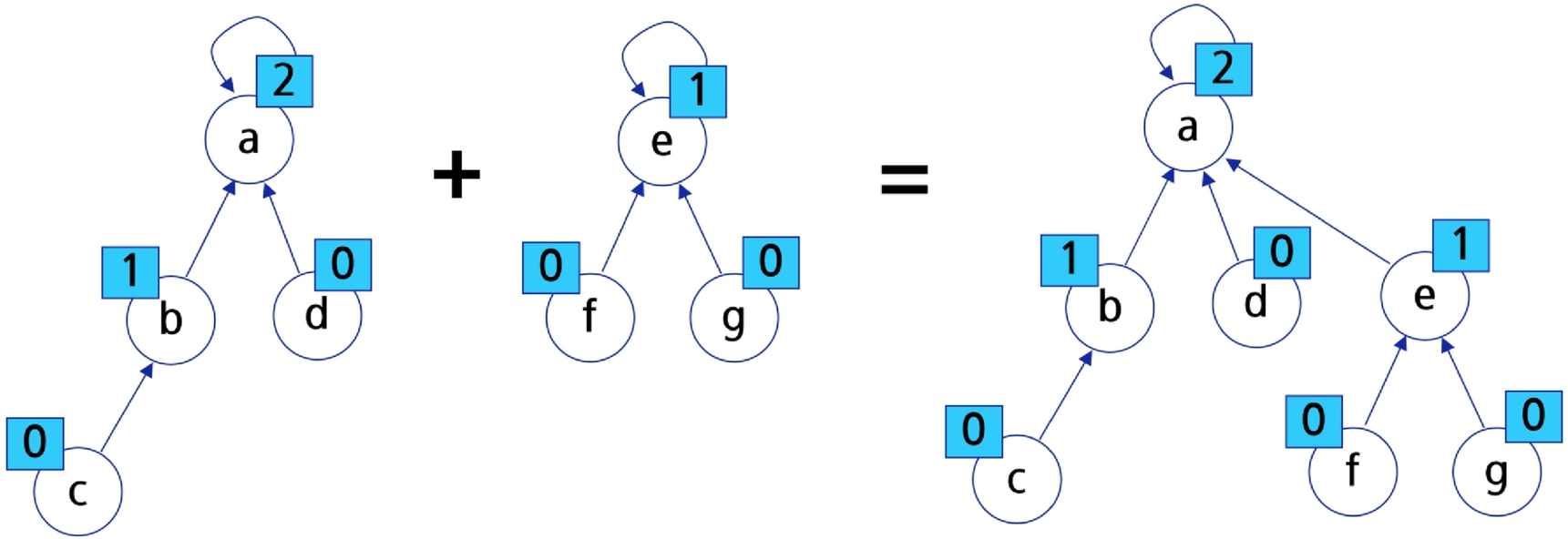


<그림28_알고리즘 응용_그래프 기본과 탐색_상호 배타 집합에 대한 연산 문제점>

연산의 효율을 높이는 방법

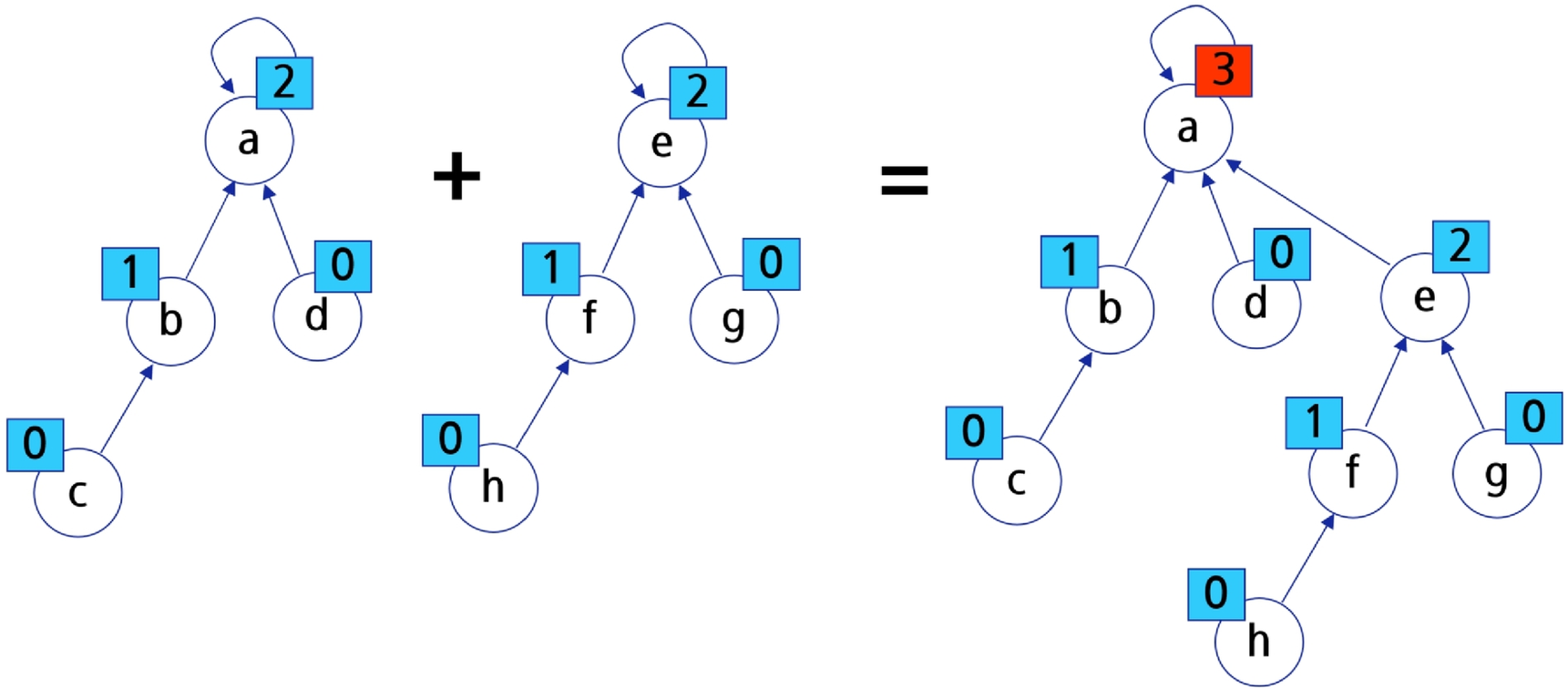
- Rank를 이용한 Union

- 각 노드는 자신을 루트로 하는 서브트리의 높이를 랭크(Rank)라는 이름으로 저장합니다.
- 두 집합을 합칠 때 rank가 낮은 집합을 rank가 높은 집합에 붙입니다.



<그림29_알고리즘 응용_그래프 기본과 탐색_랭크를 이용한 Union의 예>

랭크를 이용한 Union에서 랭크가 증가하는 예

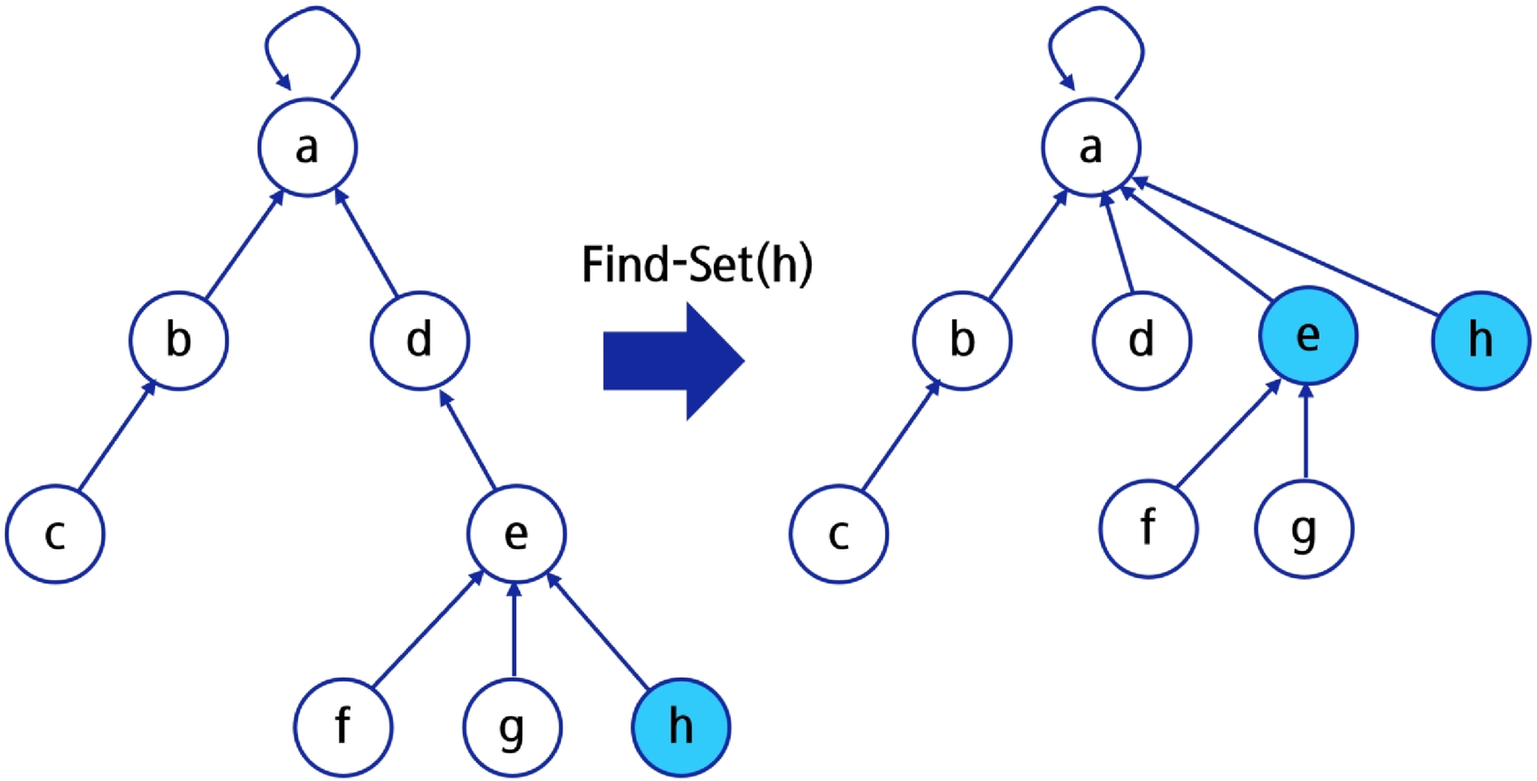


<그림30_알고리즘 응용_그래프 기본과 탐색_랭크를 이용한 Union에서 랭크가 증가하는 예>

연산의 효율을 높이는 방법

- Path compression

- Find-Set을 행하는 과정에서 만나는 모든 노드들이 직접 root를 가리키도록 포인터를 바꿔줍니다.



<그림31_알고리즘 응용_그래프 기본과 탐색_Path Compression의 예>

Make_Set() 연산

- Make_Set(x): 유일한 멤버 x를 포함하는 새로운 집합을 생성하는 연산

```
p[x] : 노드 x의 부모 저장
rank[x] : 루트 노드가 x인 트리의 랭크 값 저장

Make_Set(x)
    p[x] ← x
    rank[x] ← 0
```

Find_Set 연산

- Find_Set(x) : x를 포함하는 집합을 찾는 연산

```
Find_Set(x)
    IF x ≠ p[x]          // x가 루트가 아닌 경우
        p[x] ← Find_Set(p[x])
    RETURN p[x]
```

- Find_set 연산은 특정 노드에서 루트까지의 경로를 찾아가면서 노드의 부모 정보를 갱신한다.

Union 연산

- Union(x , y): x와 y를 포함하는 두 집합을 통합하는 연산

```
Union(x, y)
    Link( Find_Set(x), Find_Set(y) )
```

```
Link(x, y)
    IF rank[x] > rank[y]        // rank는 트리의 높이
        p[y] ← x
    ELSE
        p[x] ← y
    IF rank[x] == rank[y]
        rank[y]++
```

실습

DFS

- 24220. 경로의 수 (user)

BFS

- 5247. 연산

Union-Find(Disjoint Set)

- 5248. 그룹 나누기

추가 연습

- 10966. 물놀이를 가자

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 그래프(Graph)의 구성 요소로 옳지 않은 것은?

a) 정점(Vertex)
b) 간선(Edge)
c) 루트(Root)
d) 경로(Path)

2 인접 행렬(Adjacency Matrix) 방식의 단점으로 옳은 것은?

a) 희소 그래프일 경우 불필요한 메모리 낭비가 발생
b) 간선 유무 확인이 어려움
c) 정점의 차수를 쉽게 계산할 수 없음
d) 간선 추가가 불가능

3 DFS 탐색 중, 방문하지 않은 정점이 더 이상 없을 때 수행하는 동작은?

a) 큐에 새 정점을 삽입한다.
b) 마지막에 방문했던 정점으로 되돌아간다.
c) 전체 탐색을 중지한다.
d) 인접한 모든 정점을 다시 방문한다.

4 다음 질문에 답하시오. (단답형)

두 정점 간의 연결 여부를 2차원 배열로 표현한 그래프 저장 방식은 무엇인가요?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 서로소 집합(Disjoint Set)에서 Find-Set() 연산을 최적화하기 위한 방법으로 옳은 것은?

a) 힙 정렬
b) Path Compression
c) Topological Sort
d) Heapify

6 다음 질문에 답하시오.

인접 행렬과 인접 리스트의 차이점을
2가지 이상 설명하시오.

7 다음 질문에 답하시오. (단답형)

그래프에서 특정 정점과 직접 연결된 모든 정점을 무엇이라 하나요?

8 다음 중 인접 행렬(Adjacency Matrix) 방식의 특징으로 알맞은 것은?

a) 희소 그래프에서 메모리를 절약할 수 있다.
b) 인접한 정점을 찾는 데 O(1) 시간이 걸린다.
c) 두 정점 사이 간선 유무 확인 속도가 빠르다.
d) 간선을 추가할 때 복잡한 자료구조가 필요하다.

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 질문에 답하시오. (단답형)

서로소 집합(Disjoint Set)을 관리하는 주요 연산 2가지는 무엇인가요?

10 다음 중 Union-Find 자료구조가 적합하게 사용되는 문제 유형은?

a) 최단 거리 경로 탐색
b) 두 노드가 같은 그룹에 속해 있는지 확인
c) 그래프의 위상 정렬
d) BFS 탐색 순서 구하기

11 그래프의 경로(Path)와 관련하여 옳은 설명은?

a) 경로는 간선 없이 정점만으로 이루어진다.
b) 경로에는 정점을 중복해서 포함할 수 없다.
c) 경로는 항상 모든 정점을 방문해야 한다.
d) 시작 정점에서 다른 정점으로 이동하는 간선들의 순서를 말한다.

12 다음 질문에 답하시오. (단답형)

여러 번의 질문에 대해, 두 사람이 같은 그룹(연결 요소)에 속해 있는지 빠르게 판단해야 하는 경우 어떤 알고리즘/자료구조를 사용하는 것이 적절한가요?

확인문제

정답 및 해설

1. c) 루트(Root) 2. a) 희소 그래프일 경우 불필요한 메모리 낭비가 발생 3. b) 마지막에 방문했던 정점으로 되돌아간다. 4. 인접 행렬 (Adjacency Matrix) 5. b) Path Compression 6. 해설 참조 7. 인접 정점 (Adjacent vertices) 8. c) 간선 유무 확인 속도가 빠르다. 9. Find-Set, Union 10. b)두 노드가 같은 그룹에 속해 있는지 확인 11. d)시작 정점에서 다른 정점으로 이동하는 간선들의 순서를 말한다. 12. Union-Find (Disjoint Set)

1. 루트(Root)는 트리 자료구조에서만 사용하는 용어입니다.

2. 인접 행렬은 간선이 적은 경우에도 모든 정점 쌍의 정보를 저장하므로 메모리 낭비가 큽니다.

3. DFS에서 더 이상 방문할 정점이 없으면, 스택에서 이전 정점을 꺼내 되돌아갑니다.

4. 인접 행렬은 정점 간 연결 관계를 2차원 배열로 표현합니다.

5. 경로 압축(Path Compression)은 트리 높이를 줄여 탐색 속도를 빠르게 합니다.

6. 인접 행렬: 2차원 배열로 간선 정보를 저장
→ 메모리 소모 큼, 간선 유무 확인 O(1)
인접 리스트: 연결 리스트로 간선 저장
→ 메모리 효율적, 특정 간선 확인은 O(정점 차수)

7. 간선으로 직접 연결된 정점을 인접 정점이라고 합니다.

8. 인접 행렬은 O(1) 시간에 간선 유무를 확인할 수 있습니다.

9. 서로소 집합 자료구조는 집합의 대표를 찾는 Find-Set과 두 집합을 합치는 Union 연산을 제공합니다.

10. Union-Find는 서로소 집합 관리 자료구조로, 두 노드가 같은 그룹인지 빠르게 확인할 수 있습니다.

11. 경로(Path)는 시작 정점에서 다른 정점으로 이동하는 간선들의 순서를 의미합니다.

12. 연결 여부 판별은 Union-Find로 효율적으로 처리할 수 있습니다.

활동정리

오늘 공부한 내용 요약 및 정리

- **그래프(Graph)란?**
  - 정점(Vertex)과 간선(Edge)으로 이루어진 비선형 자료구조
  - 객체 간의 관계를 표현할 때 사용 (예: 지도, 네트워크)
  - N:N 관계를 나타내는 데 적합한 자료구조

- **그래프의 종류와 표현 방법**
  - **무방향 그래프**: 간선에 방향이 없음 (A-B와 B-A 동일)
  - **방향 그래프**: 간선에 방향이 있음
  - **가중치 그래프**: 간선에 가중치(비용)가 있음
  - **인접행렬**: 2차원 배열로 간선 유무를 표현
  - **인접리스트**: 각 정점마다 연결된 정점 리스트를 관리

활동정리

오늘 공부한 내용 요약 및 정리

- **그래프 탐색 방법**

1. 깊이 우선 탐색 (DFS)
 - 한 경로를 가능한 깊이까지 탐색하고, 막히면 이전 갈림길로 돌아가 탐색을 계속
 - **후입선출(LIFO)** 방식의 스택이나 스택 프레임을 사용하는 재귀호출로 구현

2. 너비 우선 탐색 (BFS)
 - 시작 정점에서 가까운 정점부터 차례대로 탐색
 - 선입선출(FIFO) 방식의 큐로 구현

활동정리

오늘 공부한 내용 요약 및 정리

- **DFS 동작 흐름**
  - 시작 정점 방문 → 인접 정점을 push → 더 이상 방문할 곳이 없으면 pop → 모든 정점 방문할 때까지 반복

- **BFS 동작 흐름**
  - 시작 정점을 큐에 enqueue → 큐에서 하나씩 꺼내 인접 정점을 enqueue → 큐가 빌 때까지 반복

- **응용 예시**
  - DFS: 미로 탐색, 경로 찾기, 연결 요소 탐색
  - BFS: 최단 거리 탐색, 전염 시뮬레이션, 레벨 탐색

활동정리

오늘 공부한 내용 요약 및 정리

개념	설명	예시
그래프(Graph)	정점(Vertex)과 간선(Edge)으로 이루어진 자료 구조	도시와 도로를 연결한 지도
정점(Vertex) / 간선(Edge)	정점은 대상(노드), 간선은 연결 관계를 의미	A-B 간선
인접행렬	2차원 배열로 간선 유무 표현	matrix[i][j] = 1
인접리스트	각 정점에 연결된 정점을 리스트로 표현	{0:[1,2], 1:[0,3]}
DFS (깊이 우선 탐색)	한 경로를 끝까지 따라간 후 돌아오는 탐색	스택 또는 재귀로 미로 찾기
BFS (너비 우선 탐색)	가까운 정점부터 차례로 탐색	큐를 이용해 최단 거리 탐색
후입선출(LIFO)	나중에 들어간 것이 먼저 나오는 자료 처리 방식	DFS에서 스택 사용

<표2_알고리즘 응용_그래프의 기본과 탐색_핵심 키워드>

활동정리

한 도시에서 전염병이 시작됐습니다.

도시마다 사람들의 왕래가 많아 전염병이 퍼질 위험이 있습니다.
아래 질문에 답해 봅시다.

1. 모든 도시가 전염될 수 있는지 확인하려면?
2. 가장 빨리 전염되는 도시 순서를 알고 싶다면?

활동정리

한 도시에서 전염병이 시작됐습니다.

도시마다 사람들의 왕래가 많아 전염병이 퍼질 위험이 있습니다.
아래 질문에 답해 봅시다.

1. 모든 도시가 전염될 수 있는지 확인하려면? 그래프 탐색 (DFS)
2. 가장 빨리 전염되는 도시 순서를 알고 싶다면? BFS (가까운 도시부터 탐색)

1. DFS는 한 도시에서 시작해 갈 수 있는 모든 경로를 깊이 따라가며 방문합니다.
   탐색이 끝난 후 모든 도시를 방문했다면 전염이 전체로 퍼질 수 있는 상태입니다.
2. BFS는 가까운 도시부터 차례로 탐색하므로, 전염이 퍼져 나가는 순서(1단계, 2단계 …)
   를 자연스럽게 알 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다